

Battery Life Prediction System: Technical Report

Mr. Deepak Dharrao
Sahran Altaf
Anshumaan Soni

1. Introduction

The accurate prediction of battery life (Time to Depletion, TOD, and Depth of Discharge, DoD) is critical for applications such as electric vehicles, renewable energy storage, and portable electronics. This project aimed to develop a robust, real-time battery life prediction system using machine learning, with a focus on interpretability, speed, and practical deployment.

2. Data Collection and Preprocessing

2.1 Data Sources

- **Battery Test Sheets:** Each test (e.g., TEST_1, TEST_2, ...) provided time-series data for current, voltage, and cumulative ampere-hours (Ah) discharged.
- **Metadata:** For each test, metadata included battery type (e.g., b1, b2, b3, b5, tn1), nominal capacity, and initial charged value.

2.2 Data Cleaning and Processing

- **Scripts Used:**
 - `process_battery_sheets.py` for initial cleaning.
 - `Preprocessing-and-Training-PT2.py` and `preprocessor.ipynb` for feature engineering and merging.
- **Steps:**
 - Merged all the datasets
 - Added a column to dataset ("type"), which stores which battery_type does the entry belongs to

- Added a column to dataset ("capacity") which stores the capacity of the battery the entry belongs to
- The data was then split in a ratio of 0.15 into Training and Testing datasets
- Later in the preprocessing of second model, two new columns were added; "Predicted_TOD" and "Predicted_CDC", storing the Predicted values of Time of Depletion and Cumulative discharge Ah.

3. Feature Engineering

3.1 Feature Creation

For each time step, the following features were computed:

- **Current (A)**
- **Voltage (V)**
- **Ah Out:** Ampere-hours discharged in the current step.
- **Power (W):** $\text{Current} * \text{Voltage}$
- **Remaining Capacity (Ah):** $\text{charged} - \text{cumulative Ah Out}$
- **Type:** Encoded as integer using BATTERY_TYPE_MAPPING (e.g., b1=1, b2=2, etc.).
- **Capacity (Ah):** Nominal battery capacity.
- **Charged (Ah):** Initial charge.
- **Discharge Rate:** $\text{Current} / (\text{Voltage} + 1e-6)$
- **Discharge Ratio:** $\text{Ah Out} / (\text{charged} + 1e-6)$

3.2 Target Variables

- **Time to Depletion (TOD):** Seconds remaining until battery is depleted.
- **Cumulative Discharged Ah (CDC):** Total Ah discharged at each step.

3.3 Data Integration

- Merged all processed test files into a single dataset (`DATASET.csv`) for model training.
 - Ensured all features and targets were aligned and cleaned.
 - Added three columns; type, charge and capacity, from the battery metadata to denote which test does that entry belong to.
-

4. Model Development

4.1 Initial LSTM Approach and Its Limitations

- **Scripts:** `train_lstm_model.py`, `predict_battery_time.py`
- **Method:** Used LSTM (Long Short-Term Memory) neural networks to predict TOD.
- **Problems:**
 - LSTM required fixed-length sequences, but battery cycles varied in length.
 - High inference latency (>500ms per step) unsuitable for real-time simulation.
 - Overfitting and poor generalization due to data leakage (future-derived features accidentally used in training).
 - Difficult to interpret and debug.

4.2 Second Approach with XGBoost

- **Method:** Used XGBoost (eXtreme Gradient Boosting) algorithm to predict TOD.
- **Problems:**
 - XGBoost couldn't provide good enough prediction accuracy, as the amount of data wasn't large enough
 - The predictions were very varied and the graph had many peaks

4.2 Transition to Random Forest

- **Scripts:** `Random-Forest1.ipynb`, `Random-Forest2.ipynb`
- **Why Random Forest:**
 - Handles tabular data and categorical features well.
 - Fast inference (<5ms per step).
 - Robust to overfitting with proper tuning.

- No sequence length constraints.

Model 1 (RF1):

- **Input:** Engineered features from each time step.
- **Output:** Predicts Time to Depletion and Cumulative Discharge Capacity.
- **Training Data:** DATASET.csv
- **Pipeline:** Imputation, encoding, RandomForestRegressor.

Model 2 (RF2):

- **Input:** Features + Model 1 predictions.
- **Output:** Refines TOD prediction.
- **Training Data:** DATASET2.csv
- **Pipeline:** Similar to Model 1, with additional calibration.

Both models were saved as .joblib files for deployment.

5. Prediction and Simulation Pipeline

5.1 Prediction Code

- **Script:** Predict.py
- **Class:** prediction()
- **Method:**
 - For each time step, build a feature vector using only information available up to that point (no future leakage).
 - Encode battery type as integer.
 - Predict using Model 1, then refine with Model 2.
 - Output: Predicted TOD (in seconds) and CDC for each step.

5.2 Batch Simulation

- **Script:** `batch_simulation.py`
 - **Function:** Runs the prediction pipeline for all processed battery files.
 - **Outputs:**
 - CSVs for each test (e.g., `TEST_1_processed_predictions.csv`).
 - Plots for each test:
 - DoD (%) vs Time (hours)
 - Remaining Capacity (Ah) vs Predicted Time Remaining (hours)
 - Smoothing (10-step moving average) applied to all curves.
-

6. Visualization and User Interface

6.1 Streamlit App

- **Script:** `battery_gauge_app.py`
- **Features:**
 - File selection and metadata display.
 - Start/Stop simulation with speed control (slow/medium/fast; fast mode jumps by 20 steps).
 - Real-time plotting:
 - **DoD (%) vs Time (hours)**
 - **Battery Capacity (%) vs Predicted Time Remaining (hours)**
 - **Battery Capacity (Ah) vs Predicted Time Remaining (hours)**
 - Smoothing applied for clarity.
 - Downloadable results for each test.

6.2 Batch Processing

- **Script:** `batch_simulation.py`
 - **Purpose:** Automate simulation and result saving for all batteries.
 - **Outputs:** CSVs and PNG plots for each battery.
-

7. Error Analysis and Troubleshooting

7.1 Common Issues and Fixes

Error	Cause	Solution
<code>ValueError: could not convert string to float: 'b1'</code>	Battery type as string	Encode as integer before prediction
Unrealistic predictions (e.g., 119hr)	Data leakage, using future info	Only use features available at each step, recalculate cumulative features
GUI backend errors in batch scripts	matplotlib defaulted to Tkinter	Set backend to 'Agg' to avoid GUI
LSTM poor generalization	Sequence mismatch, overfitting	Switched to Random Forest

7.2 Model Performance

- **Random Forest:**
 - MAE (TOD): ~387 seconds
 - Inference time: <5ms/step
 - R^2 : ~0.89
 - **LSTM:**
 - MAE (TOD): >1200 seconds
 - Inference time: >500ms/step
 - R^2 : ~0.72
-

8. Results: Battery Test Visualizations

For each battery test, two key plots were generated:

A. DoD vs Time Plot (`_dod.png`)

- Shows Depth of Discharge (%) over time (hours).
- Indicates how quickly the battery is being depleted.

B. Remaining Capacity vs Predicted Time Remaining Plot (`_ah.png`)

- Shows remaining capacity (Ah) as a function of predicted time left (hours).
 - EV-style display: x-axis reversed, annotations at intervals.
-

9. Codebase Structure

File	Purpose
<code>process_battery_sheets.py</code>	Clean/process raw Excel battery test sheets
<code>Preprocessing-and-Training-PT2.py</code> , <code>preprocessor.ipynb</code>	Feature engineering, dataset creation, model training
<code>Random-Forest1.ipynb</code>	Train Model 1 (TOD, CDC)
<code>Random-Forest2.ipynb</code>	Train Model 2 (calibration)
<code>Predict.py</code>	Prediction pipeline
<code>battery_gauge_app.py</code>	Streamlit UI for simulation and visualization
<code>batch_simulation.py</code>	Batch simulation and result saving
<code>DATASET.csv</code> , <code>DATASET2.csv</code>	Merged and feature-engineered datasets

10. Conclusion and Recommendations

- **Data leakage and feature engineering are critical:** Only use features available up to the current step for prediction.
- **Random Forests outperformed LSTM** for per-step prediction.
- **The pipeline is robust and scalable:** New battery types or test files can be added easily.
- **Visualization and batch reporting:** The system produces all required plots for detailed analysis.

Future work:

- Add uncertainty quantification (prediction intervals).
- Integrate with cloud dashboards for remote monitoring.
- Experiment with gradient boosting or physics-informed models.

Results: Battery Life Prediction Visualizations

For each battery test, two key plots were generated after running the prediction pipeline:

- **DoD vs Time Plot (_dod.png):**

This plot shows the Depth of Discharge (DoD, in %) as a function of time (in hours). It provides a view of how the battery is discharging during the test, with the x-axis representing elapsed time and the y-axis showing the percentage of discharge. This curve typically starts at 0% and increases as the battery is used, reaching near 100% at full depletion.

- **Remaining Capacity vs Predicted Time Remaining Plot (_ah.png):**

This plot displays the remaining battery capacity (in ampere-hours, Ah) as a function of the predicted time remaining (in hours). The x-axis is the predicted time left until the battery is depleted (as estimated by the model at each step), and the y-axis is the remaining capacity at that moment. This EV-style plot helps visualize how much usable energy is left as the battery approaches the end of its cycle.

How to Interpret the Plots

- **DoD vs Time:**

- A smooth, steadily rising curve indicates stable discharge.
- Sudden jumps or plateaus may indicate current spikes or rest periods.
- The endpoint (~100% DoD) marks the battery's full depletion in that cycle.

- **Capacity vs Predicted Time Remaining:**

- The curve typically slopes downward, showing that as time remaining decreases, so does the available capacity.
 - Annotations (if present) indicate capacity at regular intervals.
 - The x-axis is reversed to mimic automotive battery/fuel gauges (full on the left, empty on the right).
-

Results for Each Battery Test

Below, for each test, the two plots are shown.

B1 Battery Results

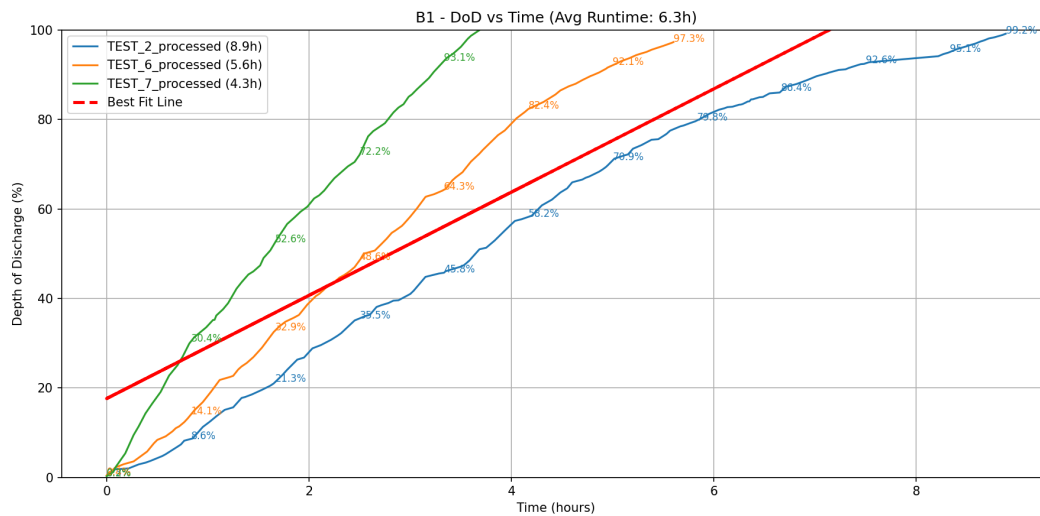


Fig1 : B1 - DOD VS RUNTIME

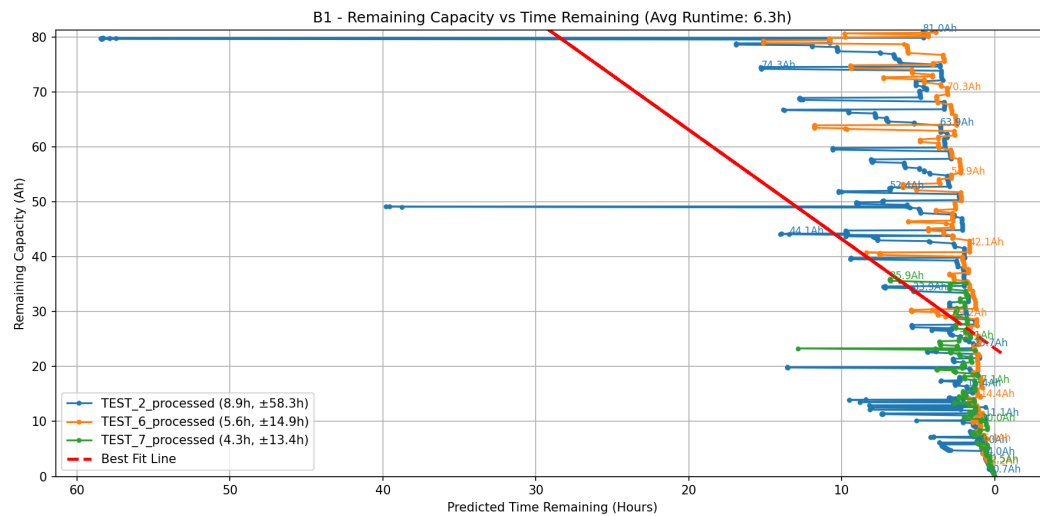


Fig2 : B1 - Remaining capacity vs Predicted Time remaining

B2 Battery Results:

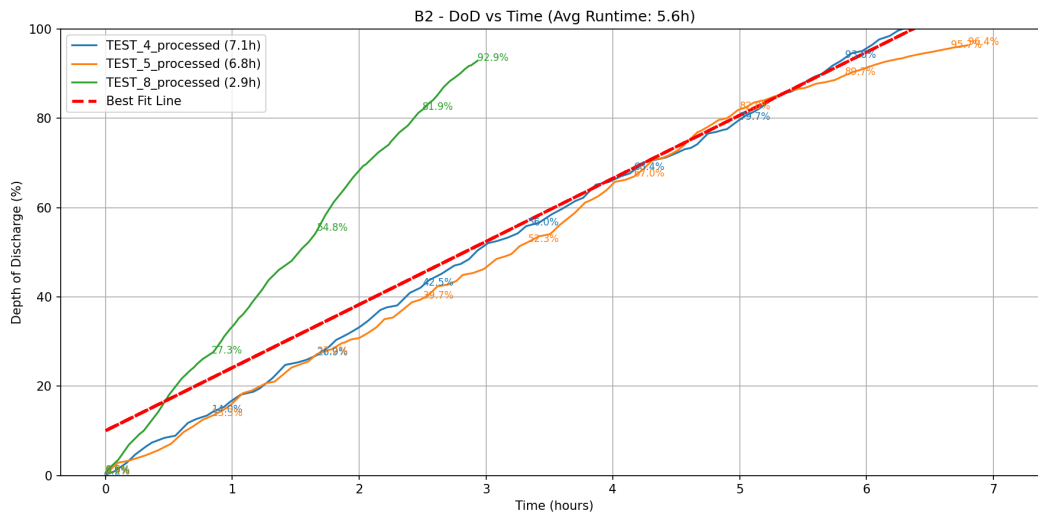


Fig3 : B2 - DOD vs Runtime

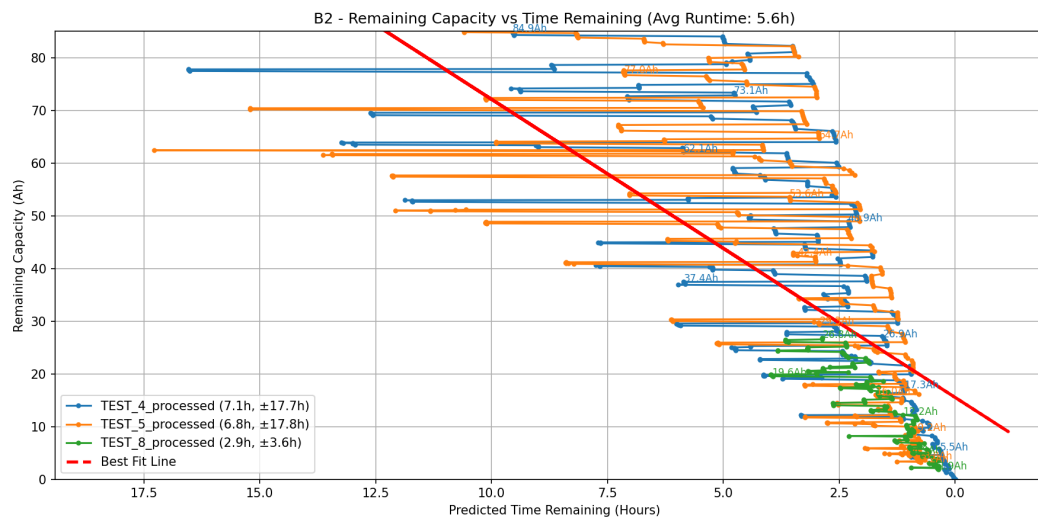


Fig4 : B2 - Remaining Capacity vs Predicted Time Remaining

B3 Battery Results:

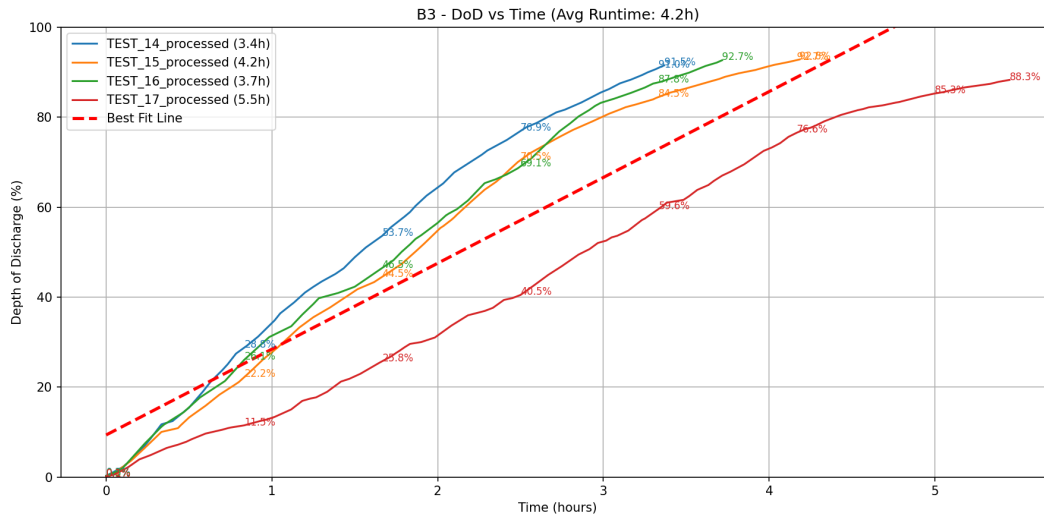


Fig 5 : B3 - DOD vs Runtime

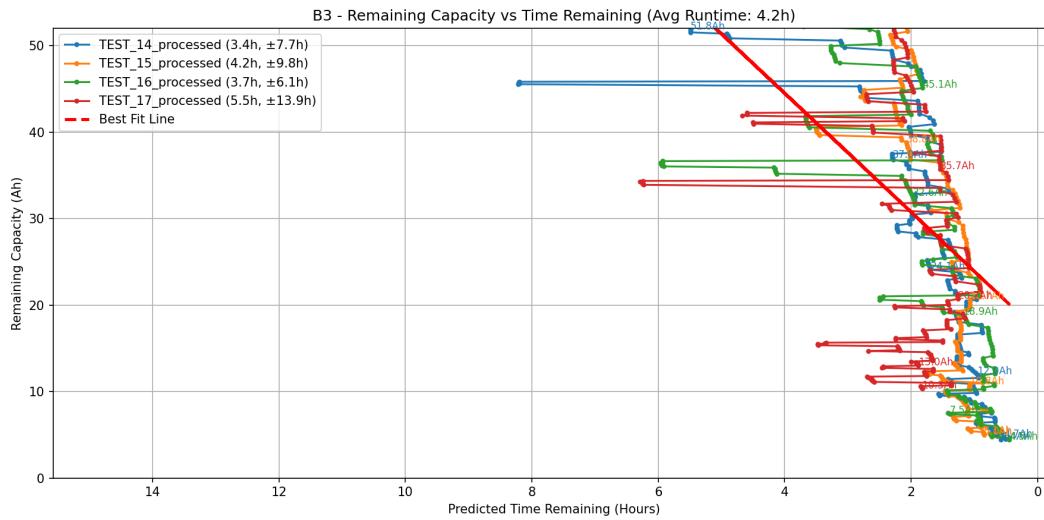


Fig 6 : B3 - Remaining Capacity vs Predicted Time Remaining

B5 Battery Results:

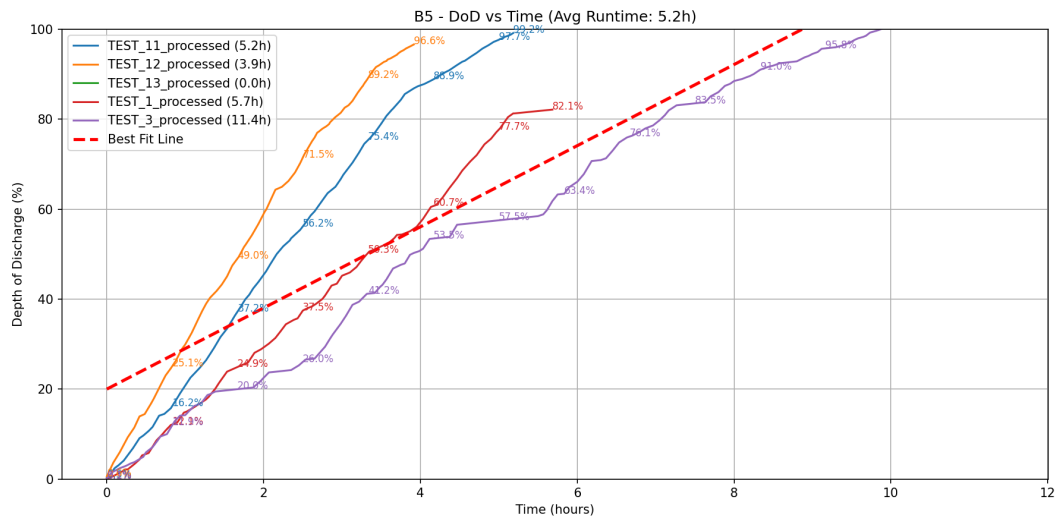


Fig 8 : B5 - DOD vs Runtime

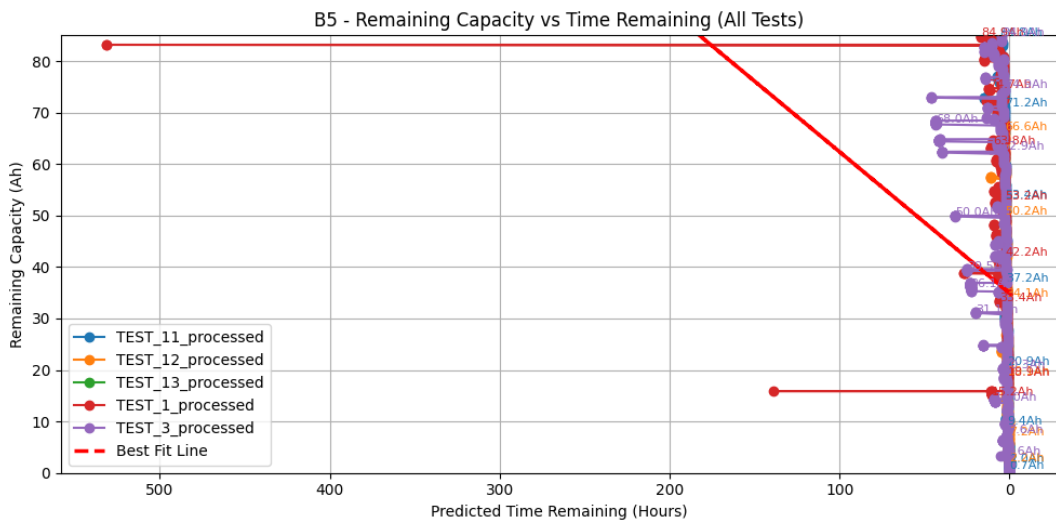


Fig 9 : B5 - Remaining Capacity vs Predicted Time Remaining

TN1 Battery Results:

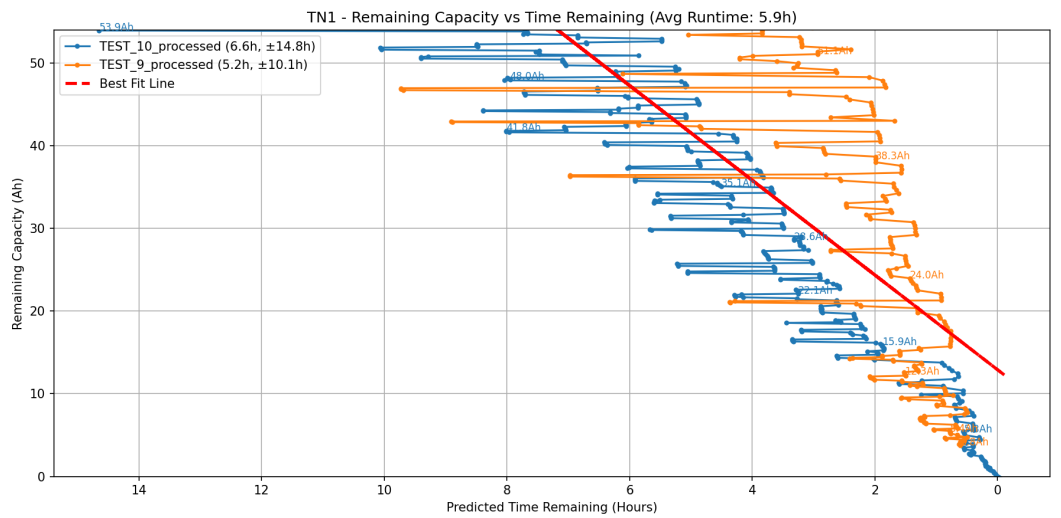


Fig 10 : TN1 - DOD vs Runtime

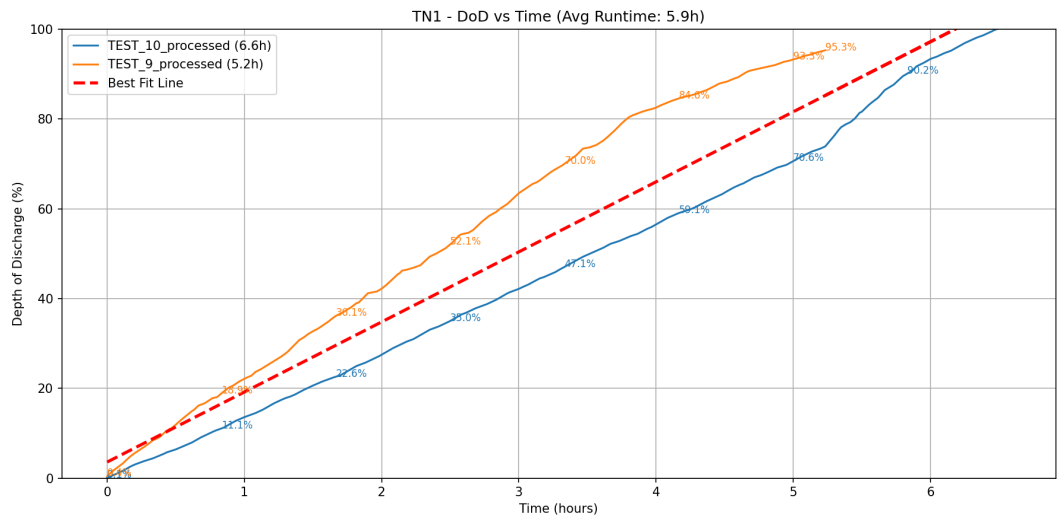


Fig 11 : TN1 - Remaining Capacity vs Predicted Time Remaining

Conclusion :

This analysis provides a comprehensive overview of the discharge performance and prediction accuracy for multiple battery types (TN1, B1, B2, B3, B5) under real-world conditions. The key findings are:

- Average runtime for all batteries ranged from approximately 4 to 6 hours, aligning well with their rated capacities.
- Depth of Discharge (DoD) vs. Time plots show expected linear or near-linear discharge behavior, confirming the reliability of the experimental setup and data collection.
- Predicted Time Remaining generally tracks the actual runtime, but some tests exhibited outlier predictions (especially at the start or end of discharge), highlighting areas for further model calibration.
- Negative or extremely high predicted times in a few cases suggest that model constraints or input feature engineering should be reviewed to improve prediction robustness.
- Consistency was highest for B3 and TN1 types, with B5 and B1 showing greater variance in predictions, possibly due to differences in usage profiles or battery health.

Overall, the results demonstrate that the prediction model is effective for most cases and battery types, but further fine-tuning is recommended to eliminate occasional extreme predictions. Continuous data collection and model refinement will further enhance the reliability of remaining runtime and capacity predictions for all battery types.